

Concurrency Program Report

For my implementation of the concurrency task scheduler, I favored a signals-based approach for each of my spawned threads. All threads are managed by main at the start and at the end as they need to be set up properly then cleanly shut down. Since the threads must be concurrent and individually spawned, this means that there must be an order to the startup process to prevent deadlock/race scenarios. Therefore, my approach is to start the channels first, spawn the workers, manager, monitor and task generator in that exact order.

For the synchronization strategy, I chose a message passing strategy as the threads all talk to each other via different channels with the main delegating when to shut down once all the tasks have been completed. While this approach doesn't use mutexes, it still allows for concurrency to work sufficiently enough. However, I do agree that some improvements by using mutexes in the architecture could improve resiliency since it's possible this approach could introduce issues I may not be aware of like race conditions.

Scheduling policy is based on both the FIFO approach, taking a naïve approach which places tasks in queue distributing them evenly between I/O and CPU, then a brute-force approach which packs most tasks depending on available compute resources.

```
==== Experiment A – Balanced / FIFO ====
Total completed:      1000
  CPU tasks:          299
  IO tasks:           701
  Makespan:         39.161088802s
  Avg wait time:       9.079653996s
  Max wait time:       18.660873555s
  Avg turnaround:      9.279955941s
  Avg CPU usage:       89.5%
  Avg active workers:  5.12 (utilization 63.9%)
  Queue length:       avg 231.61, max 469
```

The first sim test was the one where the worker idle percentage was the most noticeable with it hovering close to 64% meaning that at least for a good 40% or more of that time our workers just sat there idle, burning a good deal of work efficiency. This of course also reflects that it was also one of the slower simulations runs as well due to the under-utilization of compute resources. Interestingly though, the CPU utilization does sit fairly high at 89% which may be due to the CPU scheduling itself or other outside factors out of the realm of this test.

```
==== Experiment B – Stressed / FIFO ====
Total completed:      1000
  CPU tasks:          193
  IO tasks:           807
  Makespan:         33.088937154s
  Avg wait time:       5.96147565s
  Max wait time:       12.60746581s
  Avg turnaround:      6.161742467s
  Avg CPU usage:       89.8%
  Avg active workers:  6.05 (utilization 75.7%)
  Queue length:       avg 179.40, max 368
```

Simulation B or Stressed FIFO demonstrated improvements although still with about 1/4th of the worker time being underutilized. Regardless, this simulation test does show good improvements despite the overall worker idle time. Interestingly, CPU usage doesn't

change much, sitting at a difference of 0.4%. Even then, the time difference of average wait time from 9 seconds down to 5 is impressive for a change from naïve brute-force scheduling.

```
===== Experiment C – Stressed / Optimize =====  
Total completed:      1000  
  CPU tasks:          193  
  IO tasks:           807  
Makespan:             33.843654573s  
Avg wait time:         4.026379108s  
Max wait time:         13.45402852s  
Avg turnaround:        4.226672031s  
Avg CPU usage:         87.9%  
Avg active workers:    5.94 (utilization 74.0%)  
Queue length:          avg 118.52, max 278
```

The optimized approach is quite interesting. The overall average worker wait time does sink however the average CPU usage actually falls from 89% down to 87%. This may be due to the “packing” of the tasks as they can differ depending on how the optimization of the tasks is spread to fully “pack” the CPU utilization. Regardless, this does demonstrate that the more efficient approach of scheduling tasks to workers does lead to more efficient use of CPU resources with it being closer to the brute force approach than the slower naïve approach.